

Homework 7: Variational inference

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyana Slavchev
Görkem Kadir Solun

```
set.seed(42)
```

Problem 17: Derivation variational inference updating equation for a negative binomial mixture of cell populations

Part a: Write down the full joint probability & log-probability of the data and latent variables.

Let $x = (x_1, \dots, x_N)$, $z = (z_1, \dots, z_N)$ with $z_i \in \{1, \dots, K\}$, mixing proportions $\pi = (\pi_1, \dots, \pi_K)$, and component parameters $p = (p_1, \dots, p_K)$. The model is:

$$p(x_i \mid z_i = k, p_k) = \binom{x_i + r - 1}{x_i} p_k^r (1 - p_k)^{x_i}, \quad p(z_i = k \mid \pi) = \pi_k,$$

with priors

$$\pi \sim \text{Dirichlet}(\alpha_{0,1}, \dots, \alpha_{0,K}), \quad p_k \sim \text{Beta}(a_0, b_0).$$

Hence the full joint factorization is

$$p(x, z, \pi, p) = p(\pi) \prod_{k=1}^K p(p_k) \prod_{i=1}^N p(z_i \mid \pi) p(x_i \mid z_i, p).$$

Using one-hot indicators $\mathbb{1}[z_i = k]$:

$$p(x, z, \pi, p) = \frac{1}{B(\alpha_0)} \prod_{k=1}^K \pi_k^{\alpha_{0,k}-1} \prod_{k=1}^K \frac{1}{B(a_0, b_0)} p_k^{a_0-1} (1-p_k)^{b_0-1} \prod_{i=1}^N \prod_{k=1}^K \left[\pi_k \binom{x_i + r - 1}{x_i} p_k^r (1-p_k)^{x_i} \right]^{\mathbb{1}[z_i=k]}.$$

Therefore, the log-joint is

$$\begin{aligned} \log p(x, z, \pi, p) &= -\log B(\alpha_0) + \sum_{k=1}^K (\alpha_{0,k} - 1) \log \pi_k \\ &\quad + \sum_{k=1}^K \left[-\log B(a_0, b_0) + (a_0 - 1) \log p_k + (b_0 - 1) \log(1 - p_k) \right] \\ &\quad + \sum_{i=1}^N \sum_{k=1}^K \mathbb{1}[z_i = k] \left[\log \pi_k + \log \binom{x_i + r - 1}{x_i} + r \log p_k + x_i \log(1 - p_k) \right]. \end{aligned}$$

Part b: Derive the variational update for $Q(z_i)$

Use mean-field:

$$Q(\pi, p, z) = Q(\pi) \prod_{k=1}^K Q(p_k) \prod_{i=1}^N Q(z_i),$$

with $Q(\pi) = \text{Dirichlet}(\alpha)$ and $Q(p_k) = \text{Beta}(a_k, b_k)$.

For $Q(z_i)$:

$$\log Q^*(z_i = k) \propto \mathbb{E}_{Q(\pi)Q(p_k)} [\log \pi_k + r \log p_k + x_i \log(1 - p_k)].$$

Using Dirichlet/Beta expectations,

$$\mathbb{E}[\log \pi_k] = \psi(\alpha_k) - \psi\left(\sum_{j=1}^K \alpha_j\right),$$

$$\mathbb{E}[\log p_k] = \psi(a_k) - \psi(a_k + b_k), \quad \mathbb{E}[\log(1 - p_k)] = \psi(b_k) - \psi(a_k + b_k),$$

so responsibilities are

$$\rho_{ik} := Q(z_i = k) \propto \exp \left\{ \psi(\alpha_k) - \psi \left(\sum_{j=1}^K \alpha_j \right) + r [\psi(a_k) - \psi(a_k + b_k)] + x_i [\psi(b_k) - \psi(a_k + b_k)] \right\},$$

then normalize:

$$Q(z_i = k) = \frac{\rho_{ik}}{\sum_{\ell=1}^K \rho_{i\ell}}, \quad \sum_{k=1}^K Q(z_i = k) = 1.$$

Part c: Derive the variational update for Q[(pi)]

Collect terms involving π :

$$\log Q^*(\pi) \propto \sum_{k=1}^K \left(\alpha_{0,k} - 1 + \sum_{i=1}^N \mathbb{E}[\mathbb{1}[z_i = k]] \right) \log \pi_k.$$

Since $\mathbb{E}[\mathbb{1}[z_i = k]] = Q(z_i = k)$, we get

$$Q(\pi) = \text{Dirichlet}(\alpha_1, \dots, \alpha_K),$$

$$\alpha_k = \alpha_{0,k} + \sum_{i=1}^N Q(z_i = k).$$

Part d: Derive the variational update for Q(p_k)

Collect terms involving one p_k :

$$\log Q^*(p_k) \propto \left(a_0 - 1 + r \sum_{i=1}^N \mathbb{E}[\mathbb{1}[z_i = k]] \right) \log p_k + \left(b_0 - 1 + \sum_{i=1}^N x_i \mathbb{E}[\mathbb{1}[z_i = k]] \right) \log(1 - p_k).$$

Thus

$$Q(p_k) = \text{Beta}(a_k, b_k),$$

$$a_k = a_0 + r \sum_{i=1}^N Q(z_i = k), \quad b_k = b_0 + \sum_{i=1}^N Q(z_i = k) x_i.$$

These are the CAVI updates used iteratively with the $Q(z_i)$ update until convergence of the ELBO (or parameter change threshold).

Problem 18: R-Implementation of variational inference updating equations for a Negative Binomial mixture of cell populations

Part a: Implement the above derived variational inference procedure to estimate k and for the data in `vi.csv`.

We use the parameterization

$$X \mid z_i = k \sim \text{NegBin}(r, p_k), \quad \mathbb{E}[X \mid z_i = k] = \lambda_k = \frac{r(1 - p_k)}{p_k}.$$

```
# Load data: second column contains ATP6V0E1 counts
dat <- read.csv("vi.csv")
x <- dat[[2]]
N <- length(x)
cat("N =", N, " | mean =", round(mean(x), 3), " | var =", round(var(x), 3), "\n")
```

```
N = 20275 | mean = 0.968 | var = 1.117
```

```
# ELBO computation for the NB mixture CAVI
compute_elbo <- function(x, resp, alpha, a, b, alpha0_vec, a0, b0, r, K) {
  N <- length(x)

  # E[log p(pi)] - E[log q(pi)]
  alpha_sum <- sum(alpha)
  alpha0_sum <- sum(alpha0_vec)
  elbo <- lgamma(alpha0_sum) - sum(lgamma(alpha0_vec)) +
    sum((alpha0_vec - 1) * (digamma(alpha) - digamma(alpha_sum))) -
    (lgamma(alpha_sum) - sum(lgamma(alpha)))

  # E[log p(p_k)] - E[log q(p_k)]
  for (k in seq_len(K)) {
    e_log_pk <- digamma(a[k]) - digamma(a[k] + b[k])
    e_log_lmpk <- digamma(b[k]) - digamma(a[k] + b[k])
```

```

    elbo <- elbo +
      (a0 - 1) * e_log_pk + (b0 - 1) * e_log_1mpk -
      lgamma(a0) - lgamma(b0) + lgamma(a0 + b0) -
      ((a[k] - 1) * e_log_pk + (b[k] - 1) * e_log_1mpk -
        lgamma(a[k]) - lgamma(b[k]) + lgamma(a[k] + b[k]))
  }

# E[log p(z|pi)] + E[log p(x|z,p)] - E[log q(z)]
elog_pi <- digamma(alpha) - digamma(alpha_sum)
elog_p <- digamma(a) - digamma(a + b)
elog_1mp <- digamma(b) - digamma(a + b)

for (k in seq_len(K)) {
  Nk <- resp[, k]
  elbo <- elbo + sum(Nk * elog_pi[k])
  elbo <- elbo + sum(Nk * (lchoose(x + r - 1, x) + r * elog_p[k] + x * elog_1mp[k]))
  safe_log <- ifelse(Nk > 1e-300, log(Nk), 0)
  elbo <- elbo - sum(Nk * safe_log)
}

elbo
}

# CAVI for NB mixture (vectorized)
run_nb_vi <- function(x, K = 2, r = 10, a0 = 1, b0 = 1, alpha0 = 1,
                      max_iter = 10000, tol = 1e-6, seed = 42) {
  set.seed(seed)
  N <- length(x)
  alpha0_vec <- rep(alpha0, K)

  # Random uniform initialization for q(z_i = k), row-normalized.
  resp <- matrix(runif(N * K), nrow = N, ncol = K)
  resp <- resp / rowSums(resp)

  alpha <- alpha0_vec + colSums(resp)
  a <- a0 + r * colSums(resp)
  b <- b0 + colSums(resp * x)
  converged <- FALSE
  elbo_trace <- numeric(max_iter)

  for (iter in seq_len(max_iter)) {
    resp_old <- resp

```

```

# Update q(z_i) - vectorized over all observations.
elog_pi <- digamma(alpha) - digamma(sum(alpha))
elog_p <- digamma(a) - digamma(a + b)
elog_1mp <- digamma(b) - digamma(a + b)

# log_rho is N x K: each row = elog_pi + r*elog_p + x_i*elog_1mp
log_rho <- outer(rep(1, N), elog_pi + r * elog_p) +
  outer(x, elog_1mp)
log_rho <- log_rho - apply(log_rho, 1, max)
resp <- exp(log_rho)
resp <- resp / rowSums(resp)

# Update q(pi).
alpha <- alpha0_vec + colSums(resp)

# Update q(p_k).
a <- a0 + r * colSums(resp)
b <- b0 + colSums(resp * x)

# ELBO tracking.
elbo_trace[iter] <- compute_elbo(x, resp, alpha, a, b, alpha0_vec, a0, b0, r, K)

# Convergence check on responsibilities.
delta <- max(abs(resp - resp_old))
if (delta < tol) {
  converged <- TRUE
  break
}
}

pi_hat <- alpha / sum(alpha)
p_hat <- a / (a + b)
lambda_hat <- r * (1 - p_hat) / p_hat

list(
  r = r, iterations = iter, converged = converged,
  alpha = alpha, a = a, b = b, resp = resp,
  pi_hat = pi_hat, p_hat = p_hat, lambda_hat = lambda_hat,
  elbo_trace = elbo_trace[seq_len(iter)]
)
}

```

```

# Run for r in {10, 100, 1000} with vague priors a0=b0=alpha0=1
r_values <- c(10, 100, 1000)

# increased number of iterations from proposed 1,000 to 10,000 to ensure convergence
vi_results <- lapply(r_values, function(r) {
  run_nb_vi(x, K = 2, r = r, a0 = 1, b0 = 1, alpha0 = 1,
            max_iter = 10000, tol = 1e-6, seed = 42)
})

# Report p_k, pi_k, lambda_k
summary_table <- do.call(rbind, lapply(vi_results, function(res) {
  data.frame(
    r = res$r,
    component = seq_along(res$p_hat),
    p_k = round(res$p_hat, 6),
    lambda_k = round(res$lambda_hat, 4),
    pi_k = round(res$pi_hat, 4),
    iterations = res$iterations,
    converged = res$converged
  )
})))
rownames(summary_table) <- NULL
summary_table

```

	r	component	p_k	lambda_k	pi_k	iterations	converged
1	10	1	0.906327	1.0335	0.9281	5981	TRUE
2	10	2	0.987738	0.1241	0.0719	5981	TRUE
3	100	1	0.988012	1.2133	0.6944	4192	TRUE
4	100	2	0.995904	0.4113	0.3056	4192	TRUE
5	1000	1	0.998760	1.2415	0.6615	3864	TRUE
6	1000	2	0.999566	0.4342	0.3385	3864	TRUE

From the table above it is evident that the choice of r is important. At $r = 10$ (high overdispersion), the model explains the data by isolating a very small, near-zero expression group ($\pi_2 \approx 7\%$, $\lambda_2 \approx 0.12$). However, as r increases to 100 and 1000, the parameter estimates stabilize into two well-proportioned, distinct components. The similarity between the $r = 100$ and $r = 1000$ estimates suggests the model is highly confident in this specific separation.

```

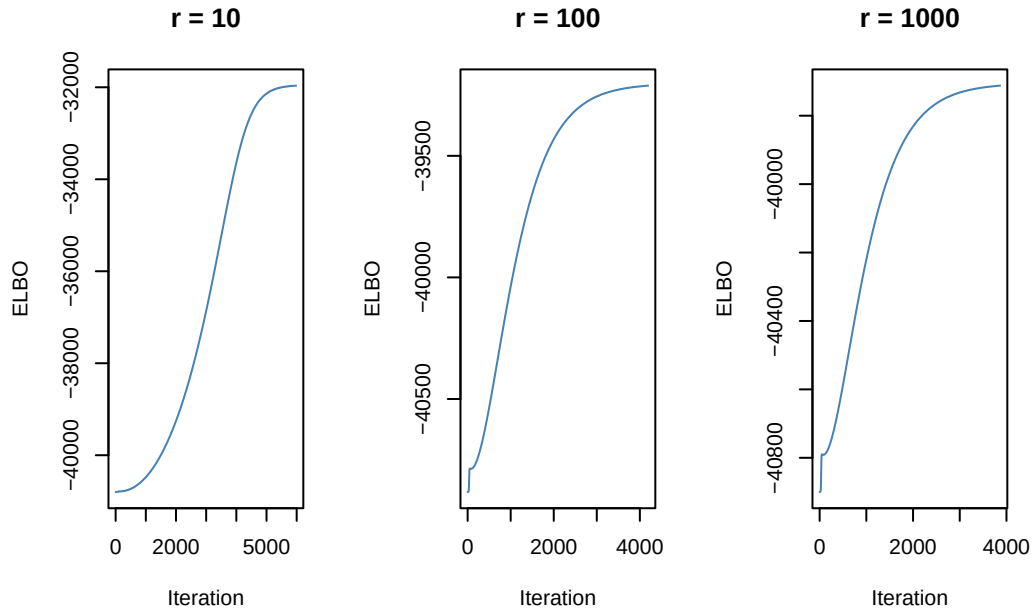
# ELBO convergence plots
par(mfrow = c(1, 3))
for (res in vi_results) {

```

```

plot(res$elbo_trace, type = "l", col = "steelblue",
     xlab = "Iteration", ylab = "ELBO",
     main = paste0("r = ", res$r))
}

```



The CAVI algorithm formally converges (`converged = TRUE`) for all values of the dispersion parameter r . The ELBO plots above confirm this success, showing a smooth, monotonic increase that plateaus, indicating the mean-field approximation has reached a stable local optimum. Interestingly, as r increases, the algorithm converges faster: around 6000 iterations at $r = 10$ to around 4000 iterations at $r = 1000$.

```

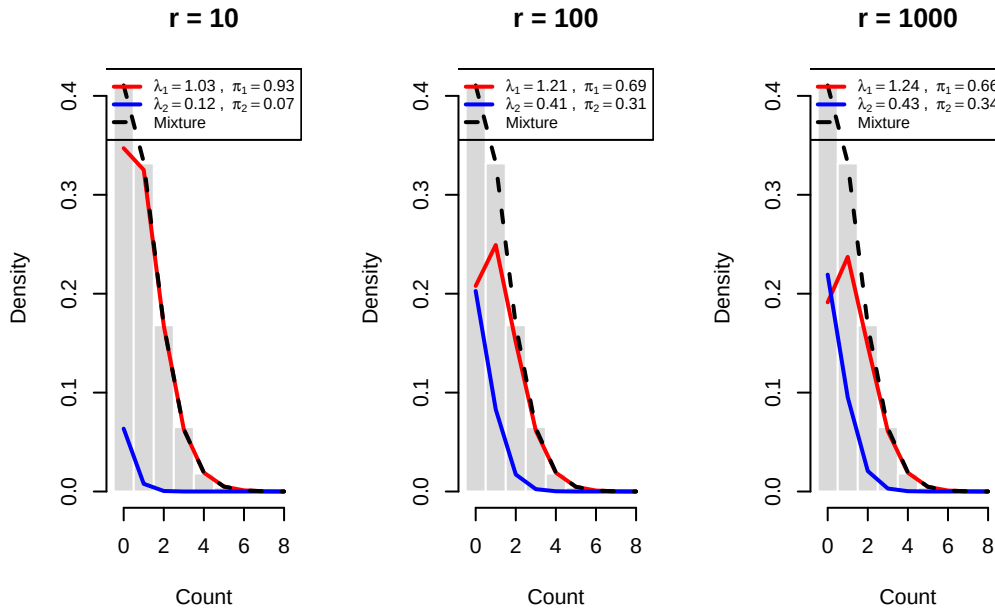
# Histogram of data with fitted NB mixture densities
par(mfrow = c(1, 3))
x_vals <- 0:max(x)
for (res in vi_results) {
  hist(x, breaks = seq(-0.5, max(x) + 0.5, by = 1), freq = FALSE,
       col = "grey85", border = "white",
       main = paste0("r = ", res$r),
       xlab = "Count", ylab = "Density")
  mix_dens <- numeric(length(x_vals))
  colors <- c("red", "blue")
  for (k in seq_along(res$p_hat)) {
    dens_k <- dnbinom(x_vals, size = res$r, prob = res$p_hat[k])

```

```

lines(x_vals, res$pi_hat[k] * dens_k, col = colors[k], lwd = 2)
mix_dens <- mix_dens + res$pi_hat[k] * dens_k
}
lines(x_vals, mix_dens, col = "black", lwd = 2, lty = 2)
legend("topright", cex = 0.7,
      legend = c(
        bquote(lambda[1] == .(round(res$lambda_hat[1], 2)) ~ ", " ~
          pi[1] == .(round(res$pi_hat[1], 2))),
        bquote(lambda[2] == .(round(res$lambda_hat[2], 2)) ~ ", " ~
          pi[2] == .(round(res$pi_hat[2], 2))),
        "Mixture"),
      col = c("red", "blue", "black"), lwd = 2, lty = c(1, 1, 2))
}

```



The plots above confirm what was observed in the results table. Unlike at $r = 10$, where there is a small, lower expression group (blue line in left plot), with $r = 100$ and $r = 1000$, the model successfully separates the data into a higher-expression group (red line, $\lambda_1 \approx 1.2$, comprising $\approx 66 - 69\%$ of cells) and a lower-expression group (blue line, $\lambda_2 \approx 0.4$, comprising $\approx 31 - 34\%$ of cells). Together, their mixture (dotted black line) matches the total distribution (bars) well. These are likely to be the two biological groups, possibly the two distinct cell types (B-cells and T-cells) characterized by their differential expression of the ATP6V0E1 gene.